

university of groningen

Performance Engineering: Lecture #11

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-05

Overview



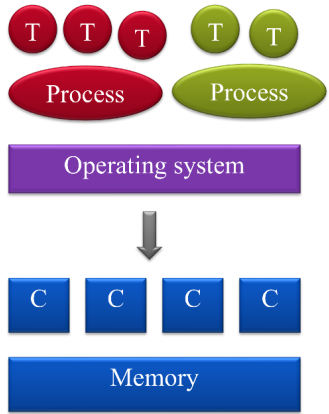
1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Shared-Memory Parallel Programming

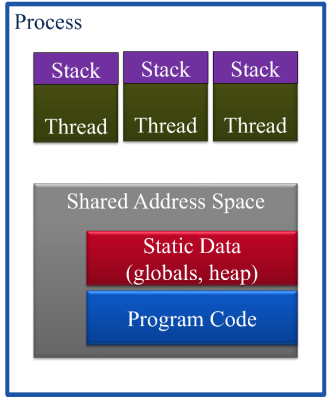


Basic assumptions:

- **Shared memory hardware support**
- An operating system that can provide
  - **Processes** with individual address spaces
  - **Threads** that share address space within a process
  - **Resource scheduling mechanisms**: thread scheduling, memory allocation and mapping

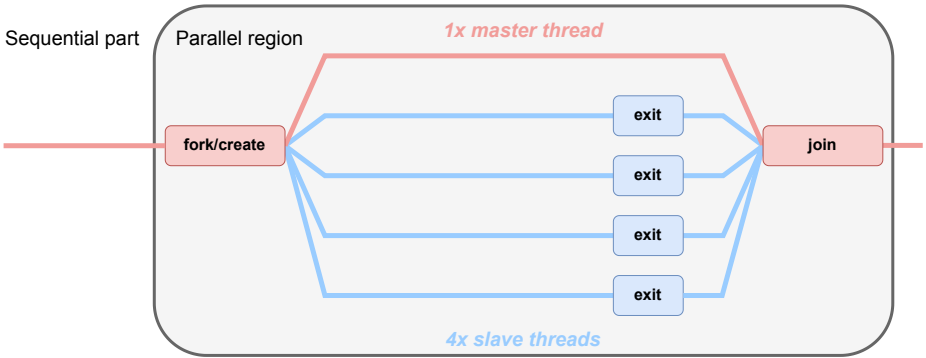
Processes versus Threads

- A process is a container for a program in execution, with state, capabilities, and access rights to resources shared by all its threads:
  - Address space
  - Code
  - Data (static data, heap data)
  - Opened files
- A thread is a unit of control flow and CPU scheduling
  - Each thread has its own PC and stack
  - Any program starts its execution as a single thread calling `main()`
  - New threads can be created through OS calls



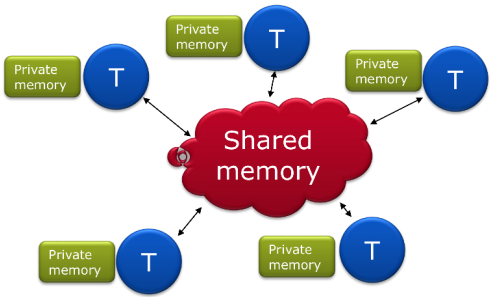
Fork-Join Model

- **Master thread** spawns a **team of threads** as needed



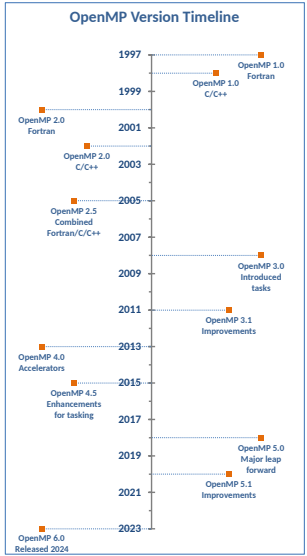
OpenMP: Introduction

- **OpenMP** = Parallel programming API
  - Based on **fork-join multi-threading model**
  - Collection of compiler directives, library routines, and environment variables
  - **Specification** defined by the **"OpenMP Architecture Review Board"**
- OpenMP requires **compiler support**
- Parallelisation is controlled by **compiler directives**
- **Abstract machine model**:
  - Concurrent threads
  - A shared address space
  - Private memory to each thread



OpenMP Evolution

- 1997: First release
- 2023: Latest release (version: 6.0)



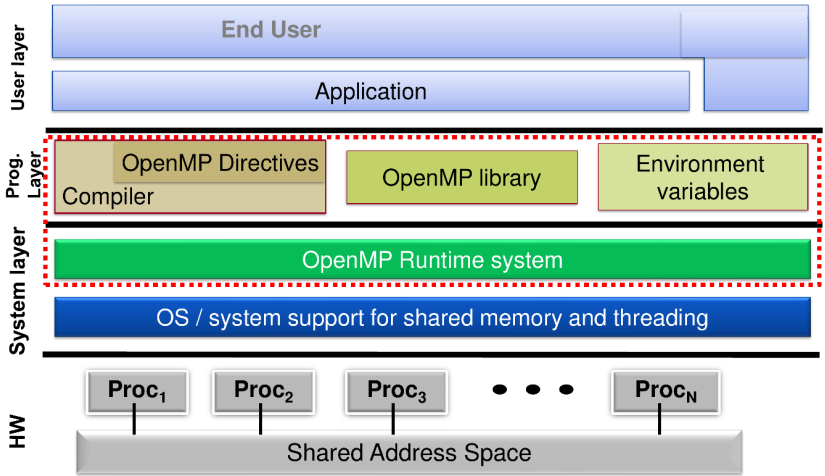
[Cornell Virtual Workshop]

Relevant Official OpenMP Documents



- Relevant documents
  - Specification “OpenMP Application Programming Interface”
  - Examples document: “OpenMP Application Programming Interface. Examples”
- Both documents are available at `https://www.openmp.org/`

OpenMP Software Stack



Number of Threads



- **Environment variable** `OMP_NUM_THREADS`
  - Sets the number of threads to use for parallel regions
  - List of positive integer values: one for each nesting level
  - Example:

    ```
    export OMP_NUM_THREADS=4,3,2
    ```
- **Function**: `void omp_set_num_threads(int)`

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

OpenMP: Core Syntax



- Most of the constructs are **compiler directives**

  `#pragma omp construct [clause [clause]...]`

  - Example

    `#pragma omp parallel num_threads(4)`
- **Function prototypes and types** in the include file `omp.h`:

  `#include <omp.h>`
- Most OpenMP constructs apply to a **structured block**
  - **Structured block**: A block of one or more statements with one point of entry at the top and one point of exit at the bottom
  - Exception: Call of `exit()` within the structured block is allowed

Hello World: Sequential Program (1/3)



```c
#include <stdio.h>


int main()
{


    int id = 0;

    printf("Hello (id=%d) ", id);
    printf("world (id=%d)\n", id);

  return 0;
}
```

Hello World: Parallel Program (2/3)



```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main()
5  {
6  #pragma omp parallel
7    {
8      int id = omp_get_thread_num();
9
10     printf("Hello (id=%d) ", id);
11     printf("world (id=%d)\n", id);
12   }
13   return 0;
14 }
```

Hello World: Compile and Run (3/3)



Compilation:

```
gcc -fopenmp hello-par.c
```

Output on Kunpeng 920:

```
Hello (id=23) world (id=11)
world (id=7)
Hello (id=20) Hello (id=25) world (id=19)
world (id=124)
Hello (id=21) Hello (id=27) world (id=22)
world (id=20)
Hello (id=28) world (id=25)
Hello (id=26) world (id=21)
world (id=6)
world (id=23)
world (id=9)
world (id=28)
world (id=26)
Hello (id=30) Hello (id=32) world (id=32)
world (id=27)
Hello (id=31) world (id=31)
world (id=30)
Hello (id=29) Hello (id=14) world (id=29)
world (id=14)
world (id=15)
Hello (id=41) world (id=41)
Hello (id=42) world (id=42)
Hello (id=44) world (id=44)
Hello (id=46) world (id=46)
Hello (id=52) world (id=52)
Hello (id=54) world (id=54)
```

Parallel Regions (1/2)

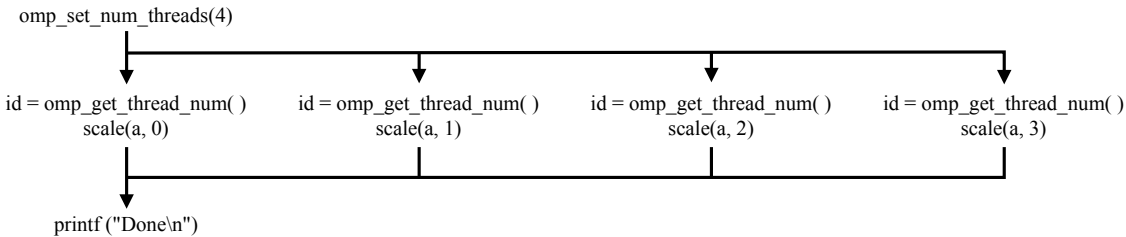


- Threads in OpenMP are created using a **`parallel` construct**
- **Example**: Parallel region with 4 threads

```
1  #include <omp.h>
2
3  double a[1000];
4
5  int main() {
6    omp_set_num_threads(4);
7
8  #pragma omp parallel
9    {
10     int id = omp_get_thread_num();
11     scale(a, id);
12   }
13
14   printf("Done\n");
15 }
```

- Each thread will call `scale(a,id)` with
  - The same value of `a`
  - Different values of `id` $= 0, 1, 2, 3$

Parallel Regions (2/2)



Shared versus Private Variables



- **Shared variables**

  `#pragma omp parallel shared (varlist)`

  - A change made to a shared variable by one thread is **eventually** visible for all threads in the program
  - Memory consistency is implementation dependent
- **Private variables**

  `#pragma omp parallel private (varlist)`

  - One local instance per thread and exclusive access by that thread
  - Allocation on the thread's run-time stack

Synchronisation: General Introduction



- Unintended sharing of data causes race conditions
  - **Race condition** = A situation in which two threads are concurrently trying to change the value of a variable
  - **No problem** if all threads do read-only accesses
  - **Problem** if read-after-write dependences exist, e.g.
    - Thread T_0 updates the shared variable x
    - Thread T_1 is expected to read the updated value of x
- Need for **synchronisation**
  - Synchronisation can be expensive ☞ **performance** engineering aims for less synchronisation
  - **Correctness** requires to implement all necessary synchronisation
    - **Beware**: The presence of race conditions is not a sufficient criterion for erroneous results

OpenMP Synchronisation



- High level synchronization
  - `critical` construct
  - `atomic` construct
  - `barrier` construct
  - `ordered` construct
- Low level synchronization flush
  - `flush` construct
  - Lock routines

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Work Scheduling



- **Challenge**: How to distribute work to the different threads?
  - Possibly work items of different size need to be distributed
  - The size of the work items might not be known at compile time
- **Scheduling strategies** implemented in OpenMP
  - **Static**: Work items are known by the programmer to be (almost) equal
  - **Dynamic**: Work items' sizes are unpredictable ☞ need for **dynamic load balancing**
    - Dynamic scheduling will come with an overhead
  - **Guided**: Special case of dynamic scheduling to reduce scheduling overhead
  - **Auto**: The run-time system tries to learn from previous executions of the same loop
- **Additional strategy**: Dynamically adjust the number of threads

`schedule` Clause



The schedule clause affects how loop iterations are mapped onto threads

- `schedule(static [,chunk])`
  - At loop entry, deal-out blocks of iterations of size `chunk` to each thread
- `schedule(dynamic[,chunk])`
  - Each thread grabs `chunk` work items (e.g., loop iterations) off a (conceptually) shared work queue, executes them and repeats, until all items have been handled
- `schedule(guided[,chunk])`
  - Dynamic scheduling where the size of the grabbed block starts large and exponentially shrinks down to minimum size `chunk` as the calculation proceeds.
- `schedule(runtime)`
  - Schedule and chunk size taken from the `OMP_SCHEDULE` environment variable
- `schedule(auto)`
  - Schedule is up to the run-time to choose (does not have to be any of the above)

Work Scheduling: Environment Variables

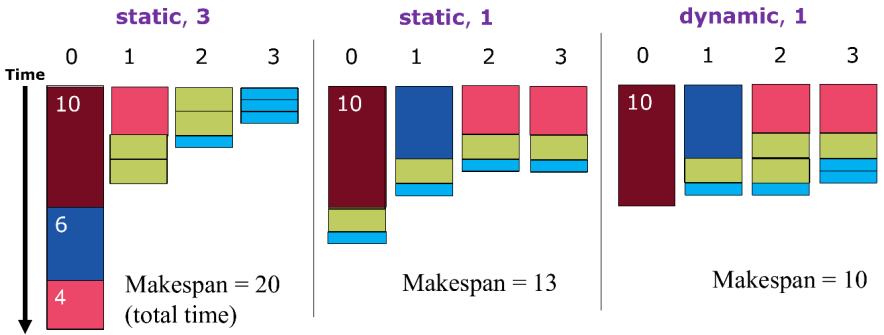


Instead of using clauses in the program, also environment variables can be used to set OpenMP **Internal Control Variables (ICV)**:

- `OMP_SCHEDULE`: `type[, chunk]`
  - Affects all OpenMP parallelised loops
  - `type`: `static`, `dynamic`, `guided`, or `auto`
  - `chunk`: An optional positive integer that specifies the chunk size
- `OMP_DYNAMIC`: `true` or `false`
  - Controls dynamic adjustment of the number of threads to use for executing parallel regions

Why Use Different Scheduling Strategies?

- Example work queue with 12 entries and different execution times: $10, 6, 4, 4, 2, 2, 2, 2, 1, 1, 1, 1$
- Assume 4 threads



Threads Placement



- **Challenge**: How to allocate hardware resources to threads?
  - Placement of threads on cores
  - Use of physical memory by each thread
- **Strategy**: Define policies
  - Allow threads to be migrated dynamically or request a fix placement
  - Placement following an affinity policy
    - **Processor affinity** = A scheduling attribute that specifies a thread's affinity for a processor core for execution
  - Physical memory allocation will follow based on OS allocation rules
- **Affinity policies**
  - `master`: Policy instructs the execution environment to assign every thread in the team to the same place as the master thread
  - `close`: Assign the threads in the team to places close to the place of the parent thread
  - `spread`: Create a sparse distribution for a team of threads among the available places

Tune Threads Placement



- **OpenMP environment variables**
  - `OMP_PROC_BIND`: Defines the affinity policies and can have the following values
    - `true`: Threads should not be moved
    - `false`: Threads may be moved
    - Comma separated list of `master`, `close`, or `spread`: Define thread affinity policy
  - `OMP_PLACES`: List of locations for placing threads
- **OpenMP clauses**
  - The `proc_bind` clause can be used in a `parallel` construct

OMP_DISPLAY_ENV



- Environment variable controls on information displayed by the run-time system at the start
- Possible values: `true`, `false`, `verbose`
- Output is implementation specific

```
1  OPENMP DISPLAY ENVIRONMENT BEGIN
2    _OPENMP = '201511'
3    [host] OMP_DYNAMIC = 'FALSE'
4    [host] OMP_NESTED = 'FALSE'
5    [host] OMP_NUM_THREADS = '1'
6    [host] OMP_SCHEDULE = 'DYNAMIC'
7    [host] OMP_PROC_BIND = 'FALSE'
8    [host] OMP_PLACES = ''
9    [host] OMP_STACKSIZE = '0'
10   [host] OMP_WAIT_POLICY = 'PASSIVE'
11   [host] OMP_THREAD_LIMIT = '4294967295'
12   [host] OMP_MAX_ACTIVE_LEVELS = '1'
13   [host] OMP_NUM_TEAMS = '0'
14   [host] OMP_TEAMS_THREAD_LIMIT = '0'
15   [all] OMP_CANCELLATION = 'FALSE'
16   [all] OMP_DEFAULT_DEVICE = '0'
17   [all] OMP_MAX_TASK_PRIORITY = '0'
18   [all] OMP_DISPLAY_AFFINITY = 'FALSE'
19   [host] OMP_AFFINITY_FORMAT = 'level %L thread %i affinity %A'
20   [host] OMP_ALLOCATOR = 'omp_default_mem_alloc'
21   [all] OMP_TARGET_OFFLOAD = 'DEFAULT'
22   [host] GOMP_CPU_AFFINITY = ''
23   [host] GOMP_STACKSIZE = '0'
24   [host] GOMP_SPINCOUNT = '300000'
25 OPENMP DISPLAY ENVIRONMENT END
```

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Ad-Hoc Checklist Methodology for Loops



1. Find compute-intensive loops
2. Check if loop iterations are
   - independent, or
   - comprise a reduction operation
3. Place the appropriate OpenMP directives
4. Check performance impact

OpenMP: Parallel Loops (1/3)



- **Step #1**: Find compute-intensive loop
- **Step #2**: Check for loop carry dependences

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (2/3)



- **Step #3**: Place directives
  - `for` **construct**

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (2/3)



- **Step #3**: Place directives
  - `for` **construct**

Question: Would code run well on a NUMA architecture?

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (2/3)



- **Step #3**: Place directives
  - `for` **construct**

Question: Would code run well on a NUMA architecture?

No: Memory is touched first by master thread

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (3/3)



- **Solution**: Ensure that first access to memory is the same as later accesses
- **Note**: Need to use a memory allocator that does not initialise memory

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13 #pragma omp parallel for
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Reduction Loops (1/2)



- **Challenge**: Loop carry dependence

```
1  double dot_product(double *restrict a,
2                     double *restrict b,
3                     int n)
4  {
5    double s = 0.0;
6
7
8    for (int i = 0; i < n; i++)
9      s += a[i] * b[i];
10
11   return s;
12 }
```

OpenMP: Reduction Loops (2/2)



- **Solution**: Use reduction clause

```
1  double dot_product(double *restrict a,
2                     double *restrict b,
3                     int n)
4  {
5    double s = 0.0;
6
7  #pragma omp parallel for reduction (+:s)
8    for (int i = 0; i < n; i++)
9      s += a[i] * b[i];
10
11   return s;
12 }
```

OpenMP Nested Loops (1/4)



- Consider addition of two matrices
- **Challenge**: How to parallelise nested loops?

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (2/4)



- **Solution #1**: Parallelise inner loop

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8  #pragma omp parallel for
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (2/4)



- **Solution #1**: Parallelise inner loop

Question: Is there a multi-threading overhead problem?

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8  #pragma omp parallel for
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (2/4)



- **Solution #1**: Parallelise inner loop

Question: Is there a multi-threading overhead problem?

Yes: Enter parallel region n times

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8  #pragma omp parallel for
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (3/4)



- **Solution #2**: Parallelise outer loop

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (3/4)



- **Solution #2**: Parallelise outer loop

Question: What if $n < N_{\text{thread}}$?

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (3/4)



- **Solution #2**: Parallelise outer loop

Question: What if $n < N_{\text{thread}}$?

At most n threads will be used

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (4/4)



- **Solution #3**: Use `collapse` clause

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for collapse(2)
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. **Cache Line Optimization**
6. Final

Cache Line Optimization: Introduction



- Cache line optimization is about inefficient use of loaded cache lines
- Main issues:
  - Poor spatial data locality
  - False sharing of cache lines (multithreaded applications only)

Poor Spatial Locality (1/2)



- **Problem**: **Poor spatial data locality** leads to not all data of a loaded cache line being used
- **Example**:

```
1  #define N 8192
2
3  double a[N][N];
4  double b[N][N];
5
6  int main() {
7    for (int j = 0; j < N; j++)
8      for (int i = 0; i < N; i++)
9        b[i][j] = a[i][j];
10 }
```

- **Mitigation strategies**:
  - Change memory access pattern
  - Adjust data structures

Poor Spatial Locality (2/2)



- **Optimised example**: Change from column-major to row-major traversal

```c
#define N 8192

double a[N][N];
double b[N][N];

int main() {
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)          // Now j is fastest running index
      b[i][j] = a[i][j];
}
```

- **Note**: This may not always be possible
  - Example: Matrix-matrix multiplication
    - One matrix is traversed row-major and the other column-major (or vice versa)

Case Study: Strided Vector Copy



- **Purpose** of this case study: Measure the impact of strided memory access
- **Specification**: Perform a strided copy of an array:

$$y_i \leftarrow x_i, \quad i = 0, 1s, 2s, 3s, \ldots$$

with stride $s \geq 1$

Strided Vector Copy: Code



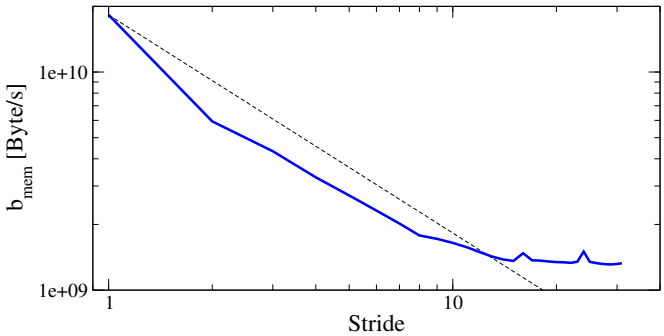
Benchmark routine:

```
1 void strided_vcpy(double *restrict y,
2                   double *restrict x,
3                   const int n,
4                   const int s)
5 {
6   for (int i = 0; i < s*n; i += s)
7     y[i] = x[i];
8 }
```

Benchmark driver:

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <time.h>
4
5 void strided_vcpy(double *restrict, double *restrict,
6                   const int, const int);
7
8 int main() {
9   int n    = 1 << 20;    int smax  = 32;
10  int nmax  = n * smax;   int nrep  = 1e3;
11  double *x = (double *) calloc(nmax, sizeof(double));
12  double *y = (double *) calloc(nmax, sizeof(double));
13
14  for (int s = 1; s < smax; s++) {
15    struct timespec t0, t1;
16    clock_gettime(CLOCK_PROCESS_CPUTIME_ID , &t0);
17
18    for (int i = 0; i < nrep; i++) strided_vcpy(y, x, n, s);
19
20    clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &t1);
21    double t  = (t1.tv_sec  - t0.tv_sec ) +
22                (t1.tv_nsec - t0.tv_nsec) * 1e-9;
23    double bw = 2 * nrep * n * sizeof(double) / t;
24    printf("%-4d %.4e\n", s, bw);
25  }
26 }
```

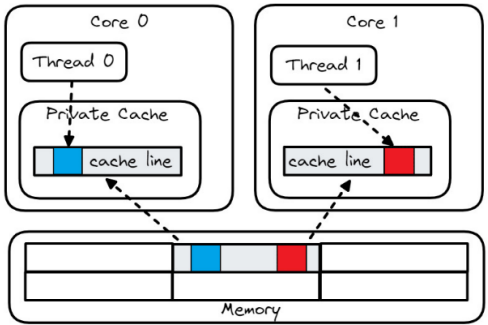
Strided Vector Copy: Results Kunpeng 920



- Until $s = 8$ one observes $b_{\mathrm{mem}}(s) \simeq b_{\mathrm{mem}}(1)/s$

False Sharing (1/2)

- **False sharing** = Two threads running on different cores modify different variables that reside on the same cache line



False Sharing (2/2)



Question: Why is false sharing problem?

False Sharing (2/2)



Question: Why is false sharing problem?

- Cores typically have private L1 data caches
- Multiple cores writing to the same cache will make the cache coherence protocol constantly moving a copy of the cache line for exclusive acces between the L1 caches

False Sharing (2/2)



Question: Why is false sharing problem?

- Cores typically have private L1 data caches
- Multiple cores writing to the same cache will make the cache coherence protocol constantly moving a copy of the cache line for exclusive acces between the L1 caches

- False versus true sharing
  - **False sharing** has only a performance impact
  - **True sharing** results in wrong results

False Sharing Example



```
struct S {
  int sumA; // sumA and sumB are likely to
  int sumB; // reside in the same cache line
};
S s;

{ // code executed by thread A
  for (int i = 0; i < N; i++)
    s.sumA += a[i];
}
```

```
{ // code executed by thread B
  for (int i = 0; i < N; i++)
    s.sumB += b[i];
}
```

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. **Final**

Summary Lecture #11



- OpenMP basics and syntax
- Performance engineering related to OpenMP
  - Optimization of critical sections
  - Tuning work scheduling and thread placement
  - Ad-hoc checklist methodology for OpenMP parallelisation of loops
  - NUMA optimization
  - Reduction loops
  - Nested loops
- Cache line optimizations: poor data locality, false sharing
- Memory-level parallelism and means to improve it

Questions?



[Wiki Media]

These slides contain content and figures from

- D. Bakhvalov, Performance Analysis and Tuning on Modern CPUs (2nd Ed.). 2024. [Online]. Available: `https://github.com/dendibakh/perf-book/releases`